Style Guidelines and Software Stack

Overview

This guide is here to help you quickly and easily get started with writing, formatting, and managing your research documents, figures, and bibliographies. By following these simple guidelines, we can foster a collaborative environment, maintain consistency and professionalism, and reduce confusion or rework for everyone—especially those new to our team.

Programming

- Anaconda/Spyder/Python:
  - Highly preferred for plotting
  - We employ Anaconda, a free and open-source distribution of Python, to streamline package management and deployment.
  - For Python development, we utilize Spyder, an open-source integrated development environment (IDE) designed for engineers and scientists.
- MATLAB with Simscape: For modeling and simulating physical systems, we employ MATLAB alongside Simscape. Simscape allows for the creation of models within the Simulink environment, facilitating the simulation of electrical circuits, mechanical assemblies, and.
- LabVIEW: LabVIEW is our chosen graphical programming environment for data acquisition, instrument control, and industrial automation. Its intuitive visual interface enables seamless integration with hardware components, streamlining the development of complex measurement and control systems.

Document Templates

- Use our lab's official repository to access PowerPoint templates, icons, and branding resources: https://github.com/ARTS-Laboratory/ARTS-Lab-Branding

Document Preparation

- LaTeX backends:
  - Windows: Use MiKTeX
  - Linux: Use TeX Live
- Editor: TeXstudio for writing and editing LaTeX documents.
- Reference Management:
  - Use JabRef for managing bibliography files (.bib).
  - Citation Key Pattern: [auth][year][shorttitle:titlecase]
  - Example: Downey2024ContinuousWaterQuality
- Use tiled "~" to create non breaking spaces between numbers and units, figures and numbers, equations and numbers, and before citations.
- Style considerations:
  - What text in equations is italic? https://physics.nist.gov/cuu/pdf/typefaces.pdf
  - To obtain non-italic units in matplotlib, consider using Unicode such as (\u03bc\u03b5) for με.

- Style for spaces around units: https://www.nist.gov/pml/special-publication-811/nist-guide-si-chapter-7-rules-and-style-conventions-expressing-values#:~:text=7.2%20Space%20between%20numerical%20value,value%20and%20the%20unit%20symbol.

- Sharing Files and Collaboration Workspace: We'll share a dedicated Dropbox folder for each publication. Dropbox makes it easy to rename and move files, helping everyone keep things organized and up to date. This also ensures that when you eventually leave the lab, you can retain your files and documents independently of university-owned systems. While Dropbox does require a subscription, it's currently the best solution we've found for maintaining a consistent workflow and enabling seamless collaboration.

Figures and Illustrations

- Software: Use Inkscape for creating and editing vector graphics and figures.
- Format: For optimal results in online publications, save figures as PNG files with a resolution of at least 300 DPI to ensure clarity and detail. If the figure contains photographic elements, using JPEG format at high-quality settings is appropriate. Avoid formats like PDF and SVG for final publication, as they can lead to larger file sizes and potential rendering issues.
- File size: To maintain control over the quality of your published work and prevent excessive compression by publishers, it's advisable to keep your entire paper's file size below 5 MB, with an absolute maximum of 10 MB. This approach minimizes the risk of the publisher applying additional compression that could degrade image quality.
- Capital letters in Figure Labels and Graphs: In our publications, we primarily use lowercase letters for labels in figures and graphs to enhance readability and maintain a clean visual presentation. This approach aligns with common practices where capitalization is reserved for proper nouns and the beginning of sentences. By using lowercase for most labels, we can effectively highlight specific elements when necessary by employing capitalization, such as for software titles or major items within a figure. This selective use of capital letters draws attention to significant components without overwhelming the reader by using bold text in figures.
- File Naming Conventions & the "making_figures" Folder: For each paper, create a dedicated folder called making_figures. Within this folder, organize subfolders named exactly after each figure as it appears in your LaTeX document (e.g., Figure1, Figure2, etc.). Each of these subfolders should contain all data files, Inkscape files, and any other assets required to produce that specific figure. This approach not only enhances reproducibility, but also makes it much easier to share or update figures in the future.
- Copy Figures into a "figures" Folder for LaTeX: Once you finalize a figure in its making_figures subfolder, place a copy of the final image file into a separate figures folder that your LaTeX document will reference. This way, you always know exactly which version of the figure is included in your paper, and you can easily provide the correct files when submitting to a journal. Keeping the "making_figures" and "figures" folders separate also serves as a helpful sanity check, ensuring you don't accidentally reference an outdated figure from somewhere else.
- No Spaces in Figure File Names: When naming your files or folders, do not use spaces. This is because many servers and systems (especially those running Linux) can encounter problems interpreting file paths with spaces, which may cause unexpected errors. Use underscores or CamelCase instead (e.g., Figure1_Draft or Figure1Draft). This practice is not only good computing hygiene but also ensures broader compatibility and fewer issues during file sharing or publication.
- Match Figure Labels to File Names: To avoid confusion and ensure consistency, use the exact same name for your figure file and the corresponding \label in LaTeX. This way, when you see

the label in your document, you'll immediately know which file it references. This practice helps prevent errors and makes it much easier to locate and update figures.

- Guidelines for Crafting Figure Captions with Subfigures:
    - Structure and Punctuation:
        - Begin with a concise description of the overall figure, followed by a colon.
        - Use lowercase letters in parentheses—(a), (b), (c), etc.—to label each subfigure.
        - Provide specific descriptions for each subfigure, separating them with commas.
        - Use "and" before the final subfigure description.
        - End the caption with a period.
    - Capitalization:
        - Start the caption with a capital letter.
        - Use lowercase for subfigure descriptions, except for proper nouns or specific terms requiring capitalization.
    - Example:
        - Microstructural analysis of alloy samples, showing: (a) as-cast microstructure, (b) microstructure after annealing, and; (c) microstructure after quenching.
            - The caption starts with an overarching description of the figure's content: "Microstructural analysis of alloy samples, showing:".
            - Each subfigure is labeled with a letter in parentheses and described succinctly.
            - Semicolons separate the subfigure descriptions, with "and" preceding the final item to delineate each component.
        - Additional Examples:
            - Stress-strain behavior of materials under different conditions, illustrating: (a) tensile test at room temperature, (b) tensile test at elevated temperature, (c) tensile test after heat treatment, and; (d) tensile test after cryogenic treatment.
            - Fatigue crack propagation in aluminum alloys, presenting: (a) crack initiation site, (b) crack growth path, (c) final fracture surface, and; (d) post-fracture microstructure.
            - Comparison of machining techniques, depicting: (a) conventional milling process, (b) laser-assisted milling, (c) ultrasonic vibration-assisted milling, and; (d) hybrid machining process.
- Color usage in plots
    - Educational Resource: It is required that you watch the following video, which provides valuable insights into effective colormap selection: Understanding Colormaps in Matplotlib. Experts have thoroughly considered this topic, so it's beneficial to leverage their findings rather than create new solutions from scratch. https://www.youtube.com/watch?v=xAoljeRJ3lU&ab_channel=Enthought
    - Sequential Data: For representing data with a natural ordering, such as intensity or magnitude, use the 'Viridis' colormap. 'Viridis' is perceptually uniform, making it a superior choice for accurately conveying data variations.
    - Categorical Data: When visualizing categorical data, utilize the 'Tab10' colormap, which is the default qualitative colormap in Matplotlib. Employing 'Tab10' ensures a consistent and distinguishable color scheme across your figures.

    - Standardized Implementation: To facilitate consistency and ease of use, a standard Python code header is provided at the end of this document. This script sets the default colormaps and handles related configurations, streamlining the plotting process.

- Font Sizes in Figure
    - Consistency with Document Text: Ensure that the font size and style in your figures match those used in your main document. Typically, this means using 10 or 11-point Times New Roman font to maintain a uniform appearance.
    - Explicit Figure Dimensions: When creating figures in Python or Inkscape, set the figure dimensions explicitly to the desired size. Import them into LaTeX using specific width settings, such as width=6.5in. Avoid using scaling commands like \textwidth=\linewidth, as they can lead to inconsistent font sizes and misaligned elements.
    - Avoid Scaling During Import: Do not scale figures up or down during the import process. At each stage—creation, export, and inclusion—consider your figure's size, resolution (DPI), and font settings. Always define these parameters explicitly to ensure clarity and consistency.
- Labeling:
    - Create all figure labels using Inkscape to ensure consistency.
    - Use white text boxes with black text for clarity.
    - Avoid adding black outlines to label boxes to maintain a clean appearance.
- Arrowheads:
    - Use triangular arrowheads to indicate flow in diagrams such as flowcharts or pipelines.
    - For labeling specific items, use arrows with rounded back ends to point to the item.
    - Ensure arrows are either white or black, depending on the figure's color scheme, to maintain visibility.
- Python Common Plotting Header

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-

import IPython as IP
IP.get_ipython().run_line_magic('reset', '-sf')
#%% import modules and set default fonts and colors

"""
Default plot formatting code for Austin Downey's series of open source notes/
books. This common header is used to set the fonts and format.

Header file last updated May 16, 2024
"""

import numpy as np
import scipy as sp
import matplotlib.pyplot as plt
import matplotlib as mpl

# set default fonts and plot colors
plt.rcParams.update({'text.usetex': True})
plt.rcParams.update({'image.cmap': 'viridis'})
plt.rcParams.update({'font.serif':['Times New Roman', 'Times', 'DejaVu Serif',
 'Bitstream Vera Serif', 'Computer Modern Roman', 'New Century Schoolbook',
 'Century Schoolbook L',  'Utopia', 'ITC Bookman', 'Bookman',
 'Nimbus Roman No9 L', 'Palatino', 'Charter', 'serif']})
plt.rcParams.update({'font.family':'serif'})
plt.rcParams.update({'font.size': 10})
plt.rcParams.update({'mathtext.rm': 'serif'})
# I don't think I need this next line as its set to 'stixsans' above.
plt.rcParams.update({'mathtext.fontset': 'custom'})
cc = plt.rcParams['axes.prop_cycle'].by_key()['color']
```

```
## End of plot formatting code

plt.close('all')
```

3/25/2025